

JS

JAVASCRIPT

COMPLETE NOTES FOR PLACEMENT

From Basics to Advanced

1. INTRODUCTION

- JavaScript (JS) is a high-level, interpreted programming language.
- It is the language of the web. It makes webpages interactive.
- Runs on all modern browsers and also on Node.js.

2. VARIABLES & DATA TYPES

Variables are containers to store data values.

Type	Example	Description
Number	123, 12.5	Number
String	"Hello"	String
Boolean	true, false	Boolean
Undefined	undefined	Value not assigned
Null	null	Intentional no value
BigInt	123n	For big integers
Symbol	Symbol('id')	Unique & Immutable

3. OPERATORS

Operators are used to perform operations on values.

Arithmetic	+ - * / % ** ++
Assignment	= += -= *= /= **=
Comparison	== != === !== > < >= <=
Logical	&& ! << >>
Bitwise	& ^ ~ << >> >>=
Ternary	condition ? expr1 : expr2
Typeof	typeof a
Instanceof	arr instanceof Array

4. STRINGS

Strings are immutable sequences of characters.

```
let s = "Hello JavaScript";
s.length // 16
s.toUpperCase() // "HELLO JAVASCRIPT"
s.toLowerCase() // "hello javascript"
s.trim() // remove spaces
s.slice(0,5) // "Hello"
s.substring(0,5) // "Hello"
s.replace("Hello","Hi") // "Hi JavaScript"
s.split(" ") // ["Hello","JavaScript"]
s.includes("Java") // true
s.startsWith("Hello") // true
s.endsWith("Script") // true
s.repeat(2) // "Hello JavaScriptHello JavaScript"
// #12 // "Hello JavaScriptHello JavaScript"
```

5. NUMBERS & MATH

```
let n = 7.89;
Math.abs(n) // 7.89
Math.round(n) // 8
Math.floor(n) // 7
Math.ceil(n) // 8
Math.max(1,5,9) // 9
Math.min(1,5,9) // 1
Math.random() // 0 to 1
Math.pow(2,3) // 8
Math.sqrt(64) // 8
Number("123") // 123
parseFloat("12.3px") // 12.3
isNaN("abc") // true
```

9. CONTROL FLOW

if, else, switch, loops control the flow.

```
if (x>10) { ... } else { ... }

switch(day) {
  case 1: ...; break;
  case 2: ...; break;
  default: ...
}

for(let i=0; i<5; i++) {
  while(condition) { }
  do { ... } while(condition);
  for (let val of arr) { } // for...of
  for (let key in obj) { } // for...in
  break; continue;
}
```

10. FUNCTIONS

Functions are reusable blocks of code.

```
// Declaration
function add(a,b){ return a+b; }

// Expression
const sub = function(a,b){ return a-b; }

// Arrow Function
const mul = (a,b) => a*b;

// Default Parameter
function greet(name="Guest"){ return `Hi ${name}`; }

// Rest Parameter
function sum(...nums){ return nums.reduce((a,b)=>a+b,0); }

// Callback Function
function timer(cb){ setTimeout(cb,1000); }
timer(()=>console.log("Done"));
```

11. DESTRUCTURING

Extract values from arrays or objects.

```
// Array
const [x,y,...rest] = [1,2,3,4]; // x=1, y=2, rest=[3,4]

// Object
const {name, age, ...other} = user;
```

12. TEMPLATE LITERALS

Use backticks ` ` for multiline strings & interpolation.

```
let name = 'Aman';
console.log(`Hello ${name}`);
```

13. DATE & TIME

```
let d = new Date();
d.getDay() // 0(Sun)-6
d.getFullYear() // 2024
d.getHours() // 0-23
d.getMonth() // 0-11
d.getMinutes() // 0-59
d.getSeconds() // 0-59
d.getDate() // 1-31
d.toISOString()
d.toLocaleString();
```

14. MATH OBJECT

Built-in math constants & functions.

```
Math.PI // 3.14159
Math.E // 2.718...
Math.pow(2,3) // 8
Math.sqrt(16) // 4
Math.random(); // 0-1
```

15. ARRAY METHODS EXAMPLS

```
nums=[1,2,3,4,5];
nums.map(x=>x*10) // [10,20,30,40,50]
nums.filter(x=>x%2===0) // [2,4]
nums.reduce((a,b)=>a+b,0) // 15
nums.find(x=>x>3) // 4
nums.some(x=>x>3) // true
nums.every(x=>x>3) // true
nums.flatMap(x=>[x,x*2]) // [1,2,2,4,3,6,4,8,5,10]
```

16. TRUTHY & FALSY

Falsy: false, 0, -0, 0n, '', null, undefined, NaN
Truthy: all others

38. TRILERIT LOOP (HOW JS WORKS)

```
graph TD
    CS[Call Stack] --> EL((Event Loop))
    EL --> CBQ[Callback Queue]
    EL --> MCQ[Microtask Queue (Promises)]
    CBQ --> EL
    MCQ --> EL
```

- Code runs in Call Stack.
- Webtasks => Callback Queue
- Callbacks go to Callback Queue.
- Promises go to Microtask Queue.

↳ console logs to Call Stack when empty.

17. ES6+ FEATURES

- let, const (block scoped)
- Arrow Functions
- Template Literals
- Destructuring
- Default Parameters
- Rest/Spread Operator
- Classes
- Modules
- Promises
- Map, Set, WeakMap, WeakSet
- Symbols
- Optional Chaining (?.)
- Nullish Coalescing (??)
- for...of loop

19. SPREAD & REST

```
// Spread (expands)
let a=[1,2,3], b=[...a,4,5]; // [1,2,3,4,5]
let obj2 = {...obj, age:23};

// Rest (collects)
function sum(...nums){... }
```

19. CLASSES (OOP)

```
class Person {
  constructor(name, age){
    this.name = name;
    this.age = age;
  }
  greet(){ return `Hi ${this.name}`; }
}

let p = new Person("Aman", 22);
p.greet();

// Inheritance
class Student extends Person{
  constructor(n,a,age,roll){
    super(n,a,age);
    this.roll = roll; }
}
```

20. THIS KEYWORD

- Promise represents that owns the currently executing code.
- In object method → object
- In function → depends on call
- In arrow function → lexical (from parent)

```
const obj = {
  name: 'Aman',
  say(){ console.log(this.name); }
}
obj.say(); // 'Aman'
```

21. CLOSURES

- A closure gives you access to an outer function's scope from an inner function.

```
function outer(x){
  return function inner(y){
    return x+y;
  }
}

const fn = outer(5);
fn(3); // 8
```

22. HOISTING

- var declarations are hoisted and initialized with undefined.
- let/const are hoisted but not initialized.
- Function declarations are fully hoisted.

```
console.log(a); // undefined
var a = 10;
```

24. MAP (KEY-VALUE STORE)

Map holds key-value pairs and remembers the insertion order.

Key	Value
'name'	'Alice'
1	100
{id:1}	'Object Key'

```
const m = new Map();
m.set('name','Alice');
m.set(1,100);
m.set({id:1},'Object Key');
```

METHODS: m.get('name'), m.has('name'), m.delete(1), m.size, m.clear()

ITERATION: for (const [k,v] of m){ console.log(k, 'x', v); }

25. SET (UNIQUE VALUES)

Set stores unique values of any type.

```
const s = new Set();
s.add(1); s.add(2);
s.add(2); // ignored
s.add('A');

s.has(1) // true
s.delete(1)
s.size // 2
s.clear()
```

ITERATION: for (const val of s){ console.log(val); }

26. PROMISES

A Promise represents the eventual completion (or failure)

```
let p = new Promise((resolve, reject)=>{
  let success = true;
  if(success) resolve('Done');
  else reject('Failed');
});
p.then(res=>console.log(res))
.catch(err=>console.log(err))
.finally(()=>console.log('Finished'));
```

27. ASYNC / AWAIT

Async function sugar over Promises.

```
async function fetchData(){
  try{
    let res = await fetch('url');
    let data = await res.json();
    console.log(data);
  }catch(err){ console.log(err); }
}
fetchData();
```

28. ERROR HANDLING

```
try {
  // risky code
} catch(err){
  console.log(err.message);
} finally {
  // always runs
}

throw new Error("Something went wrong");
```

29. REGEX (PATTERN MATCHING)

```
let re = /abc/; // /i ignore case
re.test('Abc'); // true
'abbc'.match(re); // ['abbc']
'abc'.replace(re,'X'); // 'X'
'abc'.search(re); // 0
```

37. FETCH API

Used to make HTTP requests.

```
fetch('https://api.com/data')
.then(res=>res.json())
.then(data=>console.log(data))
.catch(err=>console.log(err));

// Post Request
fetch('url',{
  method: 'POST',
  headers: {'Content-Type':'application/json'},
  body: JSON.stringify({name: 'Aman'})
})
.then(res=>res.json())
.then(console.log);
```

38. EVENT LOOP (HOW JS WORKS)

- Code runs in Call Stack.
- Web APIs handle async tasks.
- Callbacks go to Callback Queue.
- Promises go to Microtask Queue.
- Event Loop pushes tasks to Call Stack when empty.

31. DOM (DOCUMENT OBJECT MODEL)

DOM represents the structure of an HTML document.

SELECT ELEMENTS

```
document.getElementById('id')
document.querySelector('.class')
document.querySelectorAll('p')
```

CREATE & ADD

```
let div = document.createElement('div');
div.innerText = 'Hello';
document.body.appendChild(div);
```

MODIFY

```
el.style.color = 'red';
el.classList.add('active');
el.innerHTML = '<b>Hi</b>';
```

REMOVE

```
el.remove();
el.parentNode.removeChild(el);
```

32. EVENTS

Events are actions detected by the browser.

```
addEventListener('click', function(){
  console.log('Clicked');
});
```

Common Events: click, dblclick, mouseover, mouseout, keydown, keyup, submit, input, change.

33. LOCAL & SESSION STORAGE

Store data in the browser (string only).

Local (no expiry)

```
localStorage.setItem('name','Aman');
localStorage.getItem('name');
localStorage.removeItem('name');
```

Session (clears on tab close)

```
sessionStorage.setItem('id','123');
```

34. JSON

JSON is a data format.

```
let obj = {name: 'Aman', age:22};
let str = JSON.stringify(obj);
let data = JSON.parse(str);
```

35. TIMERS

```
setTimeout(()=>{ }, 1000);
clearTimeout(id);
setInterval(()=>{ }, 1000);
clearInterval(id);
```

36. MODULES (ES6)

```
// export.js
export const PI = 3.14;
export function add(a,b){ return a+b; }

// import.js
import {PI, add} from './export.js';
import addDefault from './file.js';
```

INTERVIEW TIPS

- Revise core concepts deeply
- Solve problems on arrays, strings, objects
- Understand async, promises, event loop
- Practice DOM manipulations
- Build small projects
- Stay calm and explain your thought process

Practice Regularly • Build Projects • Solve Problems • Revise & Stay Consistent - All the Best for Your Placement

DBMS (DATABASE MANAGEMENT SYSTEMS)

Cheat Sheet

DBMS is a software system that allows users to define, create, maintain, and control access to the database. It acts as an interface between the user and the data.

Goal: Efficient data management with security, consistency, and availability.

1. DATA MODELS

- Hierarchical – Tree structure
- Network – Graph structure
- Relational – Table (row-column) based
- ER Model – Entities, Attributes, Relationships
- Object-Oriented – Objects as data

2. DATABASE LANGUAGES

DDL – Define structure
CREATE, ALTER, DROP, TRUNCATE

DML – Manipulate data
INSERT, UPDATE, DELETE, SELECT

DCL – Control access
GRANT, REVOKE

TCL – Transaction control
COMMIT, ROLLBACK, SAVEPOINT

3. SQL BASICS

- SELECT – Retrieve data
- INSERT – Add data
- UPDATE – Modify data
- DELETE – Remove data
- FROM – Specify table
- WHERE – Condition
- ORDER BY – Sort
- GROUP BY – Group rows
- HAVING – Filter groups

7. JOINS

Type	Description
INNER JOIN	Returns matching rows in both tables
LEFT JOIN	All rows from left + matched from right
RIGHT JOIN	All rows from right + matched from left
FULL JOIN	All rows when there is match in either
CROSS JOIN	Cartesian product of both tables

4. CONSTRAINTS

- PRIMARY KEY – Uniquely identifies a row.
- FOREIGN KEY – References primary key of another table.
- UNIQUE – All values must be unique.
- NOT NULL – Column cannot have NULL.
- CHECK – Ensures value satisfies a condition.
- DEFAULT – Sets default value.

5. KEYS

- Super Key – Set of attributes that uniquely identify a tuple.
- Candidate Key – Minimal Super Key.
- Primary Key – Candidate key chosen to identify tuples.
- Alternate Key – Candidate keys not chosen as primary.
- Composite Key – Key formed by combination of attributes.

6. NORMALIZATION (To reduce redundancy)

- 1NF – Atomic values, no repeating groups.
- 2NF – 1NF + No partial dependency on composite key.
- 3NF – 2NF + No transitive dependency.
- BCNF – Stronger version of 3NF.

UNORMALIZED TABLE

1NF

2NF/3NF

3NF/4NF

8. AGGREGATE FUNCTIONS

- COUNT() – Number of rows
- SUM() – Sum of values
- AVG() – Average
- MIN() – Minimum value
- MAX() – Maximum value
- GROUP BY – Groups rows
- HAVING – Filters groups

9. TRANSACTION MANAGEMENT (ACID)

ACID Properties:

- Atomicity – All or nothing
- Consistency – Valid state transitions
- Isolation – Transactions don't interfere
- Durability – Changes are permanent

Commands:

- COMMIT – Save changes
- ROLLBACK – Undo changes
- SAVEPOINT – Set rollback point

Common DBMS: MySQL, Oracle, SQL Server, PostgreSQL, DB2

Think in Tables. Store Smartly. Query Efficiently.

500

CN (COMPUTER NETWORKS)

Cheat Sheet

1. INTRODUCTION

- A Computer Network is a collection of interconnected devices that communicate and share resources.
- Goals: Resource sharing, Reliability, High availability, Scalability.

2. NETWORK TYPES

- By Scope: PAN, LAN, MAN, WAN
- By Architecture: Client-Server, Peer-to-Peer
- By Topology: Bus, Star, Ring, Mesh, Tree, Hybrid

3. NETWORK TOPOLOGIES

Bus

Star

Ring

Mesh

Tree

4. OSI MODEL (7 Layers)

Layer (7-1)	Function	Example Protocols
7 Application	User Interface	HTTP, FTP, SMTP, DNS
6 Presentation	Data format, Encryption	JPEG, SSL, ASCII
5 Session	Session management	NAT/BK, PGP
4 Transport	End-to-end delivery	TCP, UDP
3 Network	Logical addressing	IP, ICMP
2 Data Link	Framing, MAC	Ethernet, PPP
1 Physical	Bits transmission	Hubs, Cables, Repeater

PDUs: Data (7-3) → Segment (4) → Packet (3) → Frame (2) → Bits (1)

5. TCP/IP MODEL (4 Layers)

Application – Combines OSI (7&6)

Transport – TCP, UDP

Internet – IP, ICMP

Network Access – Ethernet, Wi-Fi

6. COMMON PROTOCOLS

Protocol	Layer	Purpose
HTTP	Application	Web communication
FTP	Application	File Transfer
SMTP	Application	Email Sending
DNS	Application	Domain Name Resolution
TCP	Transport	Reliable delivery
UDP	Transport	Fast, connectionless
IP	Internet	Logical addressing
ICMP	Internet	Error & Control
ARP	Network Access	IP to MAC mapping

7. IP ADDRESSING (IPv4)

- 32-bit address (e.g., 192.168.1.1)
- Classes:

Class	Range	Default Mask
A	1.0.0.0 – 126.255.255.255	255.0.0.0
B	128.0.0.0 – 191.255.255.255	255.255.0.0
C	192.0.0.0 – 223.255.255.255	255.255.255.0
D	224.0.0.0 – 239.255.255.255	Multicast
E	240.0.0.0 – 255.255.255.255	Reserved

Private IP Ranges:

 - 10.0.0.0 – 10.255.255.255
 - 172.16.0.0 – 172.31.255.255
 - 192.168.0.0 – 192.168.255.255

7. ROUTING

Static Routing – Manually configured

Dynamic Routing – Automatically updated (RIP, OSPF, BGP)

10. TRANSPORT LAYER

- Connection oriented, reliable, error-checked
- UDP – Connectionless, faster, no error check

11. PERFORMANCE TERMS

- Bandwidth – Max data rate
- Throughput – Actual data rate
- Latency – Delay in transmission
- Jitter – Variation in delay
- Packet Loss – Data lost

12. DEVICES

Repeater

Hub

Switch

Bridge

Router

Modem

Gateway

Connect – Communicate – Collaborate

500

REACT (LIBRARY FOR BUILDING UI)

Cheat Sheet

React is a JavaScript library for building user interfaces, especially single-page applications. It is component-based, declarative and based on Virtual DOM.

1. JSX

JSX is a syntax extension of JavaScript.

const element = <h1>Hello</h1>;

ReactDOM.render(element, document.getElementById('root'));

2. COMPONENTS

Functional Component

function Greet() { return <h1>Hello</h1>; }

Class Component

class Greet extends React.Component { render() { return <h1>Hello</h1>; } }

3. PROPS

Props are read-only.

function Welcome(props) { return <h2>Hello, {props.name}</h2>; }

Welcome name="Anan" />

4. STATE

State is mutable and managed within a component.

const [count, setCount] = useState(0);

setCount(count + 1);

5. HOOKS (React v16.8+)

useState() – State management

useEffect() – Side effects

useContext() – Consume context

useReducer() – Complex state logic

useMemo() – Memoize values

useCallback() – Memoize functions

useLayoutEffect() – Before paint

6. EVENT HANDLING

function handleClick() { alert('Clicked'); }

button onClick={handleClick} />

7. CONDITIONAL RENDERING

isLoggedin ? <dashboard> : <login>

8. LISTS & KEYS

const items = ['a', 'b', 'c'];

return { items.map((item) => <li key={item}>{item}) }

13. REACT ROUTER (v4)

<BrowserRouter>

<Route path="/" element={<Home>} />

<Route path="/about" element={<About>} />

</BrowserRouter>

14. OPTIMIZATION

React.memo() – Prevent re-render

useMemo() – Memoize expensive value

useCallback() – Memoize function

Virtual DOM – Efficient updates

15. BUILD TOOLS

Create React App

Vite

Next.js

Component – Compose – Create Amazing UIs

500

OS (OPERATING SYSTEM)

Cheat Sheet

OS is system software that manages hardware and software resources and provides common services for computer programs.

1. FUNCTIONS

- Process Management
- Memory Management
- File Management
- I/O Management
- Security
- User Interface

2. TYPES OF OS

- Batch OS
- Multiprogramming OS
- Time Sharing OS
- Distributed OS
- Network OS
- Real-time OS
- Hard Real-time
- Soft Real-time

3. PROCESS MANAGEMENT

Process: Program in execution.

PCB (Process Control Block): Contains process state, PID, PC, registers, scheduling info, etc.

States: New → Ready → Running → Ready → Terminated

Ready → Running → Ready → Terminated

4. CPU SCHEDULING ALGORITHMS

Algorithm	Type	Description
FCFS	Non-preemptive	First Come First Serve
SJF	Non-preemptive	Shortest Job First
SRTF	Preemptive	Shortest Remaining Time First
Priority	Preemptive/Non-preemptive	Highest Priority first
Round Robin	Preemptive	Time slice based
Multilevel Queue	Preemptive	Multiple ready queues
Multilevel Feedback	Preemptive	Priority + feedback

5. DEADLOCK

4 Necessary Conditions:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

Handling:

- Prevention
- Avoidance
- Detection & Recovery
- Ignore

6. MEMORY MANAGEMENT

- Contiguous Allocation
- Fixed Partition
- Variable Partition
- Non-contiguous Allocation
- Segmentation
- Segmentation with Paging
- Virtual Memory
- Demand Paging
- Page Replacement Algorithms (FIFO, LRU, Optimal)

7. FILE MANAGEMENT

- File Concept
- File Attributes (Name, Type, Size, Location)
- File Operations (Create, Delete, Read, Write)
- Directory Structure
 - Single Level
 - Two Level
 - Tree Structured
 - Acyclic Graph

8. I/O MANAGEMENT

- Buffering
- Caching
- Spooling
- Device Drivers
- Interrupts

9. SYSTEM CALLS (Interface to OS)

- Process Control
- File Management
- Device Management
- Information Maintenance
- Communication
- Protection

Manage Resources – Optimize Performance – Ensure Security

500

MySQL (DATABASE)

Cheat Sheet

MySQL is an open-source relational database management system. It uses SQL for querying and managing data.

1. SQL DATA TYPES

Type	Description	Example
INT	Integer	10
VARCHAR(N)	Variable length str	"Anan"
CHAR(N)	Fixed length str	"A"
TEXT	Large text	"Hello..."
DATE	Date (YYYY-MM-DD)	"2024-05-01"
DATETIME	Date & Time	"2024-05-01 10:30:00"
BOOLEAN	True / False	1 / 0

2. CREATE DATABASE & TABLE

CREATE DATABASE college;

USE college;

CREATE TABLE student (id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, age INT, city VARCHAR(50));

3. ALTER TABLE

ALTER TABLE student ADD email VARCHAR(100);

ALTER TABLE student MODIFY age INT;

ALTER TABLE student DROP COLUMN email;

ALTER TABLE student RENAME TO students;

4. CRUD OPERATIONS

INSERT INTO student (id, name, age, city) VALUES (1, 'Anan', 21, 'Delhi');

SELECT * FROM student;

SELECT name, age FROM student WHERE age > 18;

UPDATE student SET age = 22 WHERE id = 1;

DELETE FROM student WHERE id = 1;

5. WHERE CLAUSE OPERATORS

Operator	Description	Example
=	Equal	age = 20
!=	Not equal	age != 20
>	Greater than	age > 20
<	Less than	age < 20
>=	Greater or equal	age >= 20
<=	Less or equal	age <= 20
BETWEEN	Between a range	age BETWEEN 18 AND 25
IN	In a list	city IN ('Delhi', 'Pune')
LIKE	Pattern match	name LIKE 'AS'
IS NULL	Is NULL	email IS NULL

6. ORDER BY, GROUP BY, HAVING

SELECT * FROM student ORDER BY age DESC;

SELECT city, COUNT(*) FROM student GROUP BY city;

SELECT city, COUNT(*) FROM student GROUP BY city HAVING COUNT(*) > 1;

7. JOINS

Join	Description	Example
INNER JOIN	Behaves matching rows	SELECT * FROM a INNER JOIN b
LEFT JOIN	All from left + matches	SELECT * FROM a LEFT JOIN b
RIGHT JOIN	All from right + matches	SELECT * FROM a RIGHT JOIN b
FULL JOIN	All when match in either	SELECT * FROM a FULL JOIN b
CROSS JOIN	Cartesian product	SELECT * FROM a CROSS JOIN b

8. AGGREGATE FUNCTIONS

Function	Description	Example
COUNT()	Count rows	SELECT COUNT(*) FROM student;
SUM()	Sum of values	SELECT SUM(age) FROM student;
AVG()	Average	SELECT AVG(age) FROM student;
MIN()	Minimum value	SELECT MIN(age) FROM student;
MAX()	Maximum value	SELECT MAX(age) FROM student;

9. PRIMARY KEY & FOREIGN KEY

CREATE TABLE department (dept_id INT PRIMARY KEY, dept_name VARCHAR(50));

CREATE TABLE employee (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), dept_id INT, FOREIGN KEY (dept_id) REFERENCES department(dept_id));

10. CONSTRAINTS

- PRIMARY KEY – Uniquely identifies a row.
- FOREIGN KEY – References another table.
- UNIQUE – All values must be unique.
- NOT NULL – Column cannot be NULL.
- CHECK – Ensures condition is satisfied.
- DEFAULT – Sets default value.

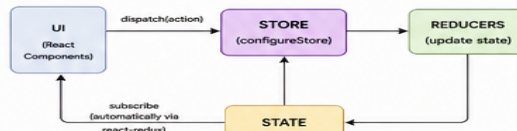
Practice SQL Daily – Think in Tables – Get Better at Queries

500

REDUX TOOLKIT (RTK) CHEAT SHEET

The official, recommended way to write Redux logic.

Quick Overview



2. STORE

The store holds the entire application state. Create Store using configureStore.

```
store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../features/counter/counterSlice';
import userReducer from '../features/user/userSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer,
  },
  // middleware, devTools are added by default
});

export default store;
```

Key Points

- configureStore automatically sets up the Redux DevTools
- It includes thunk middleware by default
- reducer object keys become state slice names

1. INTRODUCTION

Redux Toolkit (RTK) is the official, opinionated, batteries-included toolset for efficient Redux development. It simplifies Redux by:

- Providing good defaults
- Reducing boilerplate
- Using Immer for immutable updates
- Including useful utilities

RTK = createSlice + configureStore + Built-in Middleware

3. SLICE (createSlice)

A slice contains: state, reducers, actions. createSlice automatically generates action creators and action types.

```
features/counter/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  value: 0,
  status: 'idle',
  error: null,
};

const counterSlice = createSlice({
  name: 'counter', // slice name
  initialState,
  reducers: {
    increment: (state) => { // Immer allows direct mutation
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
    reset: (state) => {
      state.value = 0;
    },
  },
});

export const { increment, decrement, reset } = counterSlice.actions;
export default counterSlice.reducer;
```

What happens here?

- createSlice uses Immer internally, so you can "mutate" state safely.
- It automatically generates action creators (increment, decrement, reset).
- It automatically generates action types (counter/increment, ...).

4. DISPATCHER (What is dispatch?)

Dispatch is a function provided by the Redux store. It is used to send (dispatch) an action to the store.

When an action is dispatched:

1. It goes to the store.
2. The store passes it to the reducer.
3. The reducer updates the state.
4. The UI is re-rendered with new state.

How to use dispatch?

```
import { useDispatch, useSelector } from 'react-redux';
import { increment, decrement, reset } from '../counterSlice';

const Counter = () => {
  const dispatch = useDispatch(); // dispatcher
  const count = useSelector((state) => state.counter.value);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>+1</button>
      <button onClick={() => dispatch(decrement())}>-1</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
};
```

Key Points

- dispatch(action) sends an action to the store.
- Action can be a plain action (object) or a thunk (function).
- useDispatch() hook gives you the dispatch function.

6. SELECTORS (useSelector)

Selectors are used to read data from the Redux store.

```
import { useSelector } from 'react-redux';

const count = useSelector((state) => state.counter.value);
const users = useSelector((state) => state.user.users);
const status = useSelector((state) => state.user.status);
```

Best Practices

- Keep state minimal and normalized
- Use selectors to access only needed state
- Avoid storing derived data in Redux

7. MIDDLEWARE (Built-in & Custom)

RTK includes useful middleware by default:

- redux-thunk - for async logic
- redux-devtools - for DevTools
- serializableCheck - checks non-serializable values
- immutableCheck - checks state mutations (dev only)

You can add custom middleware.

```
const store = configureStore({
  reducer,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(logger), // custom middleware
});
```

8. ACTIONS (Types & Structure)

An action is a plain JavaScript object.

Structure:

```
{
  type: 'sliceName/actionName',
  payload: data (optional)
}
```

Example (auto-generated by createSlice):

```
{
  type: 'counter/increment'
}
```

Example (with payload):

```
{
  type: 'user/setUser',
  payload: { id: 1, name: 'John' }
}
```

9. STATE STRUCTURE

```
Store
└─ reducer (root)
   └─ counter {alice}
      └─ { value, status, error }
         └─ user {slice}
            └─ { users, status, error }

Access in component:
state.counter.value
state.user.users
```

10. IMMER (Built-in)

RTK uses Immer under the hood. Immer lets you write "mutative" code which is converted to immutable updates.

Example:

```
// You can write this
state.value += 1;

// Immer converts it to this internal
{
  ...state,
  value: state.value + 1,
};
```

Benefits

- Less boilerplate
- Cleaner code
- No manual shallow copies

13. COMPLETE EXAMPLE

Counter Slice + Store + Component

counterSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    reset: (state) => { state.value = 0; },
  },
});

export const { increment, decrement, reset } = counterSlice.actions;
export default counterSlice.reducer;
```

store.js

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

export const store = configureStore({
  reducer: { counter: counterReducer },
});
```

Counter.jsx

```
import { useDispatch, useSelector } from 'react-redux';
import { increment, decrement, reset } from './counterSlice';

const Counter = () => {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>+1</button>
      <button onClick={() => dispatch(decrement())}>-1</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
};

export default Counter;
```

15. BEST PRACTICES

1. Use RTK (configureStore + createSlice + createAsyncThunk) for all new projects.
2. Keep state normalized. Use createEntityAdapter for complex data.
3. Keep components presentational. Move logic to slices/thunks.
4. Use selectors to read only the required state.
5. Avoid storing non-serializable values in state.
6. Handle loading/error states for async operations.
7. Use TypeScript with RTK for better development experience.

16. COMMON PATTERNS

Loading State Pattern

```
InitialState: { data: [], status: 'idle', error: null }
// status: 'idle' | 'loading' | 'succeeded' | 'failed'
```

Optimistic Update Pattern

```
// update UI first
state.items.push(newItem);
try { await apiCall(); }
catch { state.items.pop(); }
```

Error Handling Pattern

```
.addCase(thunk.rejected, (state, action) => {
  state.status = 'failed';
  state.error = action.payload || 'Something went wrong';
});
```

Pagination Pattern

```
state.page += 1;
state.items.push(...action.payload);
state.hasMore = action.payload.length > 0;
```

14. DISPATCH FLOW (Step by Step)

1. Component calls dispatch(action)
2. Dispatch sends action to store
3. Store passes action to reducer
4. Reducer updates state
5. Store notifies subscribers
6. UI re-renders with new state



17. KEY TERMS

- Store** - Holds the entire state tree
- Action** - An object that describes what happened
- Dispatch** - Function to send an action to the store
- Reducer** - Function that updates the state
- Slice** - A collection of reducers and actions
- State** - The data of the application
- Thunk** - A function that can contain async logic
- Middleware** - Intercepts actions before they reach reducers

★ Redux Toolkit makes Redux simpler, faster, and more scalable!